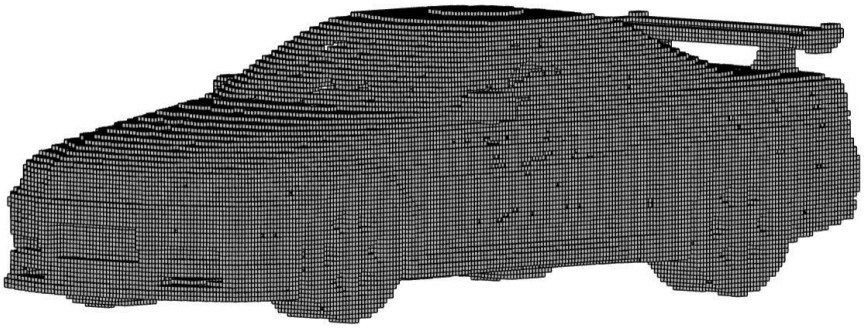
IF2211 Strategi Algoritma
Tugas Kecil 2
Voxelization Objek 3D menggunakan Octree



Dipersiapkan oleh:

13524031 Vincent Rionarlie

13524033 Ray Owen Martin

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132

2026

Daftar Isi

Daftar Isi 2
Daftar Tabel 3
Daftar Gambar 4
1 Pendahuluan 5
2 Algoritma Program 5
3 Contoh Tangkapan Layar 15
4 Checklist 17
5 Pernyataan Tidak Melakukan Kecurangan 17
6 Lampiran 17

Daftar Gambar

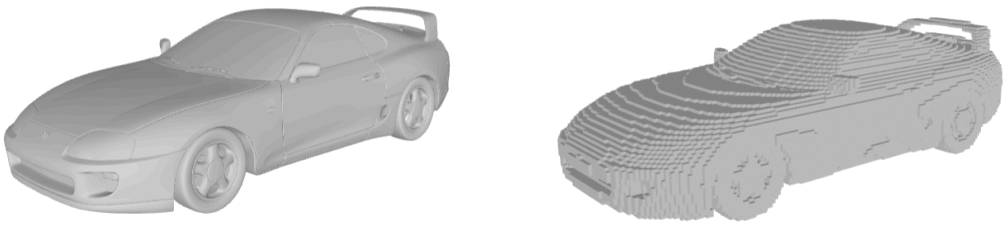
Gambar 1.1 Contoh gambar polygon mesh (kiri) dan objek voxelnya (kanan) (sumber: https://sketchfab.com/3d-models/toyota-supra-mk4-supra-1997-free-download-18e530a1f4e04b4bbc5f51a09e2f95e2; dikonversi menjadi obj dengan https://convert3d.org/gltf-to-obj/app, dikonversi menjadi voxel dengan program penulis dan divisualisasikan dengan https://3dviewer.net/#) 4
Gambar 2.1 Source Code Octree.go 5
Gambar 2.2 Source Code Parser.go(1) 5
Gambar 2.3 Source Code Parser.go(2) 6
Gambar 2.4 Source Code Face.go 7
Gambar 2.5 Source Code Point.go 8
Gambar 2.6 Source Code divcon.go (1) 8
Gambar 2.7 Source Code divcon.go (2) 9
Gambar 2.8 Source Code divcon.go (3) 10
Gambar 2.9 Source Code intersect.go (1) 11
Gambar 2.10 Source Code intersect.go (2) 12
Gambar 2.11 Source Code intersect.go (3) 13
Gambar 2.12 Source Code outhandle.go (1) 13
Gambar 2.13 Source Code outhandle.go (2) 14
Gambar 2.14 Source Code Wrapper.go 15
Gambar 2.15 Source Code main.go 16

1 Pendahuluan

Voxelization adalah suatu proses pengubahan suatu objek tiga dimensi (seperti jenis *polygon mesh*, objek yang terdiri dari banyak titik dan segitiga yang menghubungkan antar titik) menjadi suatu objek lain yang terdiri atas kubus-kubus kecil yang disebut *voxel*. Objek hasil *voxelization* mudah diimajinasikan dengan membayangkan permainan *Minecraft*, suatu permainan yang setiap bendanya tampak seperti kotak-kotak.

Proses *voxelization* ini dapat dilakukan dengan menggunakan konsep *Octree*, yaitu suatu pohon yang dapat bercabang 8 anak, lalu anaknya dapat bercabang lagi 8 anak dan seterusnya. Akar *octree* menunjukkan kubus terkecil yang dapat membungkus objek tiga dimensi *polygon mesh*, yang kemudian dibagi-bagi membentuk 8 kubus dengan rusuk berukuran setengah panjang rusuk awal, yang kemudian kubus-kubus ini dapat dibagi lagi.

Contoh *polygon mesh* dan objek voxel dapat dilihat di bawah ini:



Gambar 1.1 Contoh gambar polygon mesh (kiri) dan objek voxelnya (kanan)
(sumber: https://sketchfab.com/3d-models/toyota-supra-mk4-supra-1997-free-download-18e530a1f4e04b4bbc5f51a09e2f95e2; dikonversi menjadi obj dengan https://convert3d.org/gltf-to-obj/app, dikonversi menjadi voxel dengan program penulis dan divisualisasikan dengan https://3dviewer.net/#)

Tugas ini mengimplementasikan sebuah program yang dapat mengkonversi model .obj yang umum menjadi model .obj yang terdiri dari voxel dengan memanfaatkan properti Octree. Program melakukan konversi model dengan menggunakan algoritma divide and conquer.

2 Algoritma Program

Program ini menggunakan bahasa Go dan memiliki 6 package:

- octree : berisi definisi dan konstruktor tipe data octree
- parser : berisi cara parsing file .obj dengan format yang sesuai
- point : berisi definisi dan metode-metode tipe data point (titik) dan face (sisi) dari objek *polygon mesh*
- voxelizer : algoritma utama, berisi seluruh proses *voxelization* dari pembagian menjadi kubus-kubus, metode pembantu yang menentukan terpotong tidaknya suatu titik atau sisi pada kubus, dan metode ekspor hasil ke bentuk file .obj
- wrapper : berisi algoritma pembantu yang membungkus objek awal di .obj
- main : kode program utama yang menjalankan seluruh algoritma

Setiap package ini berisikan file-file dan algoritma berikut ini:

a. Package octree
   - Octree.go
     Source code:

```go
package octree

import (
    "tucil/src/point"
)

type OctreeNode struct {
    Children [8]*OctreeNode
    InFaces  []point.Face

    Centre   point.Point // Titik tengah kubus
    HalfSide float32     // Setengah panjang sisi kubus

    IsLeaf bool
}

func NewOctreeNode(p point.Point, s float32) *OctreeNode {
    return &OctreeNode{
        Centre:   point.Point{X: p.X, Y: p.Y, Z: p.Z},
        HalfSide: s,
        IsLeaf:   false,
    }
}
```

Gambar 2.1 Source Code Octree.go

File ini memiliki beberapa bagian:

- Definisi tipe data OctreeNode
  OctreeNode memiliki 5 atribut sebagai berikut:
  - ➢ Children : pointer ke 8 OctreeNode lain yang adalah anak dari OctreeNode ini
  - ➢ InFaces : face yang intersect dengan OctreeNode ini
  - ➢ Centre : titik tengah kubus
  - ➢ HalfSide : setengah panjang dari rusuk kubus
  - ➢ IsLeaf : boolean yang menandakan apakah suatu OctreeNode adalah daun atau bukan (berada di ujung/tidak intersect dengan titik dan sisi manapun)

- Fungsi NewOctreeNode
  Suatu konstruktor OctreeNode yang menerima parameter Point p dan float32 s serta mengembalikan suatu OctreeNode dengan titik tengah p, HalfSide s, dan isLeaf false.

b. Package parser

- Parser.go
  Source code:

```go
package parser

import (
    "bufio"
    "fmt"
    "log"
    "os"
    "strconv"
    "strings"
)

func HandleFile(path string) (res *os.File) {
    file, err := os.Open(path)
    if err != nil {
        log.Fatal(err)
    }
    return file
}
```

Gambar 2.2 Source Code Parser.go(1)

- Fungsi HandleFile
  Suatu fungsi yang membuka file sesuai parameter string path, mengembalikan kode error bila gagal membuka file, jika berhasil maka kembalikan data File.

```go
func Parse(path string) (res1 [][3]float32, res2 [][3]int) {
    file := HandleFile(path)
    defer file.Close()
    buf := bufio.NewScanner(file)
    for buf.Scan() {
        line := buf.Text()
        if strings.HasPrefix(line, "v ") {
            var a, b, c float32
            fmt.Sscanf(line, "v %f %f %f", &a, &b, &c)
            temp := [3]float32{a, b, c}
            res1 = append(res1, temp)
        } else if strings.HasPrefix(line, "f ") {
            token := strings.Fields(line)
            var temp [3]int
            for i := 0; i < 3; i++ {
                // Ambil bagian sebelum slash pertama
                a := strings.Split(token[i+1], "/")[0]
                val, err := strconv.Atoi(a)
                if err == nil {
                    temp[i] = val
                }
            }
            res2 = append(res2, temp)
        }
    }
    if err := buf.Err(); err != nil {
        log.Fatal(err)
    }

    return res1, res2
}
```

Gambar 2.3 Source Code Parser.go(2)

- Fungsi Parse
  Suatu fungsi yang menerima parameter string path dan mengembalikan array float32 dua dimensi dengan width 3 res1 dan array int dua dimensi dengan width 3 res2.
  Fungsi ini akan membaca file di path dan menyimpan semua vertex yang berformat {v (float) (float) (float)} di res1. Untuk face, fungsi Parse dapat handle kedua kondisi ketika format barisnya {f (int) (int) (int)} maupun yang berformat {f (int)/(...) (int)/(...) (int)/(...)}. Ini diperlukan ketika mencari file objek melalui sumber lain yang berwarna atau yang memiliki ekstensi file yang bukan .obj karena bagian face-nya akan berformat {f (int)/(...) (int)/(...) (int)/(...)}. Hasil parsing face ini akan disimpan di res2.

c. Package point
   - Face.go
     Source code:

```go
package point

import (
    "fmt"
)

type Face struct {
    V0 Point
    V1 Point
    V2 Point
}

func ToFace(p []Point, list [][3]int) []Face {
    res := make([]Face, 0, len(list))
    for _, f := range list {
        temp := Face{ p[f[0]-1],
                      p[f[1]-1],
                      p[f[2]-1], }
        res = append(res, temp)
    }
    return res
}

func PrintFaces(list []Face) {
    for i, f := range list {
        fmt.Printf("Face - %v\n", i+1)
        fmt.Printf("V0: %+v\n", f.V0)
        fmt.Printf("V1: %+v\n", f.V1)
        fmt.Printf("V1: %+v\n", f.V2)
        fmt.Println()
    }
}
```

Gambar 2.4 Source Code Face.go

- Definisi tipe data Face
  Tipe data Face memiliki 3 atribut, V0, V1, dan V2 yang menandakan 3 titik yang dihubungkan oleh Face ini.

- Fungsi ToFace
  Suatu fungsi yang menerima parameter array of Point p dan array of integer dua dimensi dengan width 3 list dan mengembalikan array of Face.
  Fungsi ini menerima sejumlah titik dan array yang berisi titik-titik yang terhubung untuk membuat tipe data yang lebih sesuai, yaitu sejumlah Face.

- Prosedur PrintFaces
  Suatu fungsi yang menerima parameter array of Face list dan melakukan output setiap face dengan titik-titik yang terhubung oleh Face tersebut.

- Point.go
  Source code:

```go
package point

import (
    "fmt"
)

type Point struct {
    X float32
    Y float32
    Z float32
}

func ToPoint(vert [][3]float32) []Point {
    arrP := make([]Point, len(vert))
    for i, v := range vert {
        arrP[i] = Point{v[0], v[1], v[2]}
    }
    return arrP
}

func PrintPoints(arrP []Point) {
    for v := range arrP {
        fmt.Printf("%d : %.6f %.6f %.6f\n", v, arrP[v].X, arrP[v].Y, arrP[v].Z)
    }
}
```

Gambar 2.5 Source Code Point.go

- Definisi tipe data Point
  Tipe data Point memiliki 3 atribut, X, Y, dan Z yang merupakan koordinat pada ruang dimensi tiga.

- Fungsi ToPoint
  Suatu fungsi yang menerima parameter array of Point vert dan mengembalikan array of Point.
  Fungsi ini menerima sejumlah titik untuk membuat tipe data yang lebih sesuai, yaitu sejumlah Point.

- Prosedur PrintFaces
  Suatu fungsi yang menerima parameter array of Point arrP dan melakukan output setiap face dengan titik-titik yang terhubung oleh Face tersebut.

d. Package voxelizer
   - divcon.go
     Source code:

```go
package voxelizer

import (
    "fmt"
    "math"
    "sync"
    "time"
    "tucil/src/octree"
    "tucil/src/point"
    "tucil/src/wrapper"
)

var DepthList = make(map[int]int)
var depthMu sync.Mutex

func ConcurrencyActivation(taskID int, wg *sync.WaitGroup, node *octree.OctreeNode,
depth int, targetSize float64) {
    defer wg.Done()
    Divide(node, depth, targetSize)
    fmt.Printf("Task %d done.\n", taskID)
}
```

Gambar 2.6 Source Code divcon.go (1)

- Variabel global DepthList
  Tipe map[int]int, merupakan variabel yang menyimpan jumlah node yang terpakai pada tiap level kedalaman akar Octree utama.

- Variabel global depthMu

Tipe sync.Mutex, merupakan variabel yang mengunci dan membuka kunci variabel DepthList untuk menjaga prinsip Mutual Exclusion (mutex).

- Prosedur ConcurrencyActivation
  Prosedur yang menerima parameter int taskID, pointer ke WaitGroup wg, pointer ke OctreeNode node, int depth, dan float64 targetSize.
  Prosedur ini intinya memanggil prosedur Divide dengan parameter node, depth, dan targetSize tetapi untuk concurrency, sehingga memanggil wg.Done() yang mengurangi jumlah task yang sedang jalan secara konkuren dan menandainya dengan output taskID (identifier dari task yang dibuat berjalan secara konkuren) yang sudah selesai.

```go
func StartVoxelize(points []point.Point, faces []point.Face) (*octree.OctreeNode,
time.Time) {
    root := wrapper.WrapInBox(points)

    exp := math.Log2(float64(root.HalfSide)*2) - 8.0
    fmt.Println("\nEvery voxel is represented with the size of 2 to the power of
    'i'")
    fmt.Println("Please enter your desired 'i'")
    fmt.Println("Hint : 'i' can be negative")
    fmt.Printf("Hint : default 'i' is %v\n", int(exp))
    fmt.Print(">> ")
    fmt.Scanf("%f\n", &exp)

    targetSize := math.Pow(2, exp)

    start_time := time.Now()

    root.InFaces = faces
    Divide(root, 0, targetSize)
    return root, start_time
}
```

Gambar 2.7 Source Code divcon.go (2)

- Fungsi StartVoxelize
  Fungsi yang menerima parameter array of Point points dan array of Face faces, mengembalikan pointer ke OctreeNode dan Time.
  Fungsi ini memulai proses voxelization dengan membungkus points ke dalam suatu kotak dan membuatnya menjadi suatu OctreeNode root, menerima input di terminal yang menentukan ukuran target (juga diberikan rekomendasi dengan rumus pada baris exp:= …), mulai simpan waktu saat ini dengan start_time yang nanti direturn bersama dengan root, lalu memulai proses utama voxelization yaitu dengan memanggil prosedur Divide.

```go
func Divide(node *octree.OctreeNode, depth int, targetSize float64) {
    depthMu.Lock()
    DepthList[depth]++
    depthMu.Unlock()

    if node.HalfSide <= float32(targetSize)/2 {
        node.IsLeaf = true
        return
    }

    // make children
    childHalf := node.HalfSide / 2.0
    offsets := [8][3]float32{
```

```go
        {1, 1, 1}, {-1, 1, 1}, {1, -1, 1}, {-1, -1, 1},
        {1, 1, -1}, {-1, 1, -1}, {1, -1, -1}, {-1, -1, -1},
    }

    // iterate all children
    for i := 0; i < 8; i++ {
        childCentre := point.Point{
            X: node.Centre.X + (offsets[i][0] * childHalf),
            Y: node.Centre.Y + (offsets[i][1] * childHalf),
            Z: node.Centre.Z + (offsets[i][2] * childHalf),
        }

        var childInclFaces []point.Face

        // test for any intersection
        for _, f := range node.InFaces {
            if AabbTest(childCentre, childHalf, f) {
                childInclFaces = append(childInclFaces, f)
            }
        }

        // make real children
        if len(childInclFaces) > 0 {
            node.Children[i] = &octree.OctreeNode{
                Centre:   childCentre,
                HalfSide: childHalf,
                InFaces:  childInclFaces,
                IsLeaf:   false,
            }
        }
    }

    node.InFaces = nil

    var wg sync.WaitGroup
```

```go
    if depth == 0 {
        for i := 0; i < 8; i++ {
            if node.Children[i] != nil {
                wg.Add(1)
                go ConcurrencyActivation(i, &wg, node.Children[i], depth+1,
                targetSize)
            }
        }
        wg.Wait()
    } else {
        for i := 0; i < 8; i++ {
            if node.Children[i] != nil {
                Divide(node.Children[i], depth+1, targetSize)
            }
        }
    }
}
```

Gambar 2.8 Source Code divcon.go (3)

- Prosedur Divide
  Prosedur yang menerima parameter pointer ke OctreeNode node, int depth, dan float64 targetSize.
  Prosedur ini adalah algoritma utama bertipe rekursi dan merupakan jenis Divide and Conquer yang melakukan proses pembagian Octree hingga ukuran yang diinginkan (targetSize). Karena program ini memanfaatkan konkurensi, maka program dimulai dengan mengunci DepthList agar dapat diupdate, setelah itu DepthList di-*unlock*.
  Selanjutnya program akan mengecek apakah node saat ini adalah Leaf (ukuran kubus sudah lebih kecil daripada ukuran yang diinginkan), jika iya maka hentikan prosedur ini, jika tidak maka lanjutkan ke bagian menyusun kedelapan anak.
  Bagian menyusun kedelapan anak dari node dilakukan dengan membuat posisi setiap anak berdasarkan kedelapan sudut kubus, lalu cek apakah ada irisan dengan titik atau face objek, jika ada barulah dibuat anaknya, jika tidak maka proses rekursi berhenti.
  Terakhir, jika prosedur ini dilakukan di kedalaman 0 (root OctreeNode utama), maka program langsung mengerjakan proses pembagian ke-8 anak secara konkuren, jika bukan, maka lakukan secara sekuensial.

- intersect.go
  Source Code:

```go
package voxelizer

import (
    "tucil/src/point"
)

func abs(x float32) float32 {
    if x < 0 { return -x }
    return x
}

func vecSub(a point.Point, b point.Point) point.Point {
    return point.Point{X: a.X - b.X, Y: a.Y - b.Y, Z: a.Z - b.Z}
}

func vecCross(a point.Point, b point.Point) point.Point {
    return point.Point{
        X: a.Y*b.Z - a.Z*b.Y,
        Y: a.Z*b.X - a.X*b.Z,
        Z: a.X*b.Y - a.Y*b.X,
    }
}

func vecDot(a point.Point, b point.Point) float32 {
    return a.X*b.X + a.Y*b.Y + a.Z*b.Z
}
```

Gambar 2.9 Source Code intersect.go (1)

- Fungsi abs
  Fungsi helper yang mengembalikan nilai mutlak berbentuk float, dari sebuah float *x*.
- Fungsi vecSub
  Fungsi helper yang mengembalikan hasil pengurangan dua vektor (dalam kode, vektor direpresentasikan dengan point) dalam bentuk vektor pula.
- Fungsi vecCross
  Fungsi helper yang mengembalikan hasil *cross-product* dari dua vektor dalam bentuk vektor. *Cross-product* mengembalikan vektor normal dari sebuah bidang yang dibentuk dua vektor.
- Fungsi vecDot
  Fungsi helper yang mengembalikan hasil *dot-product* dari dua vektor dalam bentuk vektor. *Dot-product* mengembalikan besaran proyeksi sebuah vektor pada vektor lainnya.

```go
func AabbTest(centre point.Point, halfSide float32, f point.Face) bool {
    p0 := vecSub(f.V0, centre)
    p1 := vecSub(f.V1, centre)
    p2 := vecSub(f.V2, centre)

    minF_x := min(p0.X, p1.X, p2.X)
    minF_y := min(p0.Y, p1.Y, p2.Y)
    minF_z := min(p0.Z, p1.Z, p2.Z)
    maxF_x := max(p0.X, p1.X, p2.X)
    maxF_y := max(p0.Y, p1.Y, p2.Y)
    maxF_z := max(p0.Z, p1.Z, p2.Z)

    // == Test xyz axes (cek klo si segitiga tbtb jadi kotak gitu)
    if (minF_x > halfSide || maxF_x < -halfSide) { return false }
    if (minF_y > halfSide || maxF_y < -halfSide) { return false }
    if (minF_z > halfSide || maxF_z < -halfSide) { return false }

    // == Test face's normal (cek si kotak nempel/nembus ga ama segitiga)
    l0 := vecSub(p1, p0)
    l1 := vecSub(p2, p1)
    l2 := vecSub(p0, p2)

    n := vecCross(l0, l1)
    cf := vecDot(n, p0) // centre to face
    cc := halfSide * (abs(n.X) + abs(n.Y) + abs(n.Z)) // centre to corner

    if (cf > cc || cf < -cc) { return false }
```

Gambar 2.10 Source Code intersect.go (2)

- Fungsi AabbTest

  Fungsi ini sangat penting dalam penyusunan algoritma proyek ini. Dalam fungsi *divide and conquer*, sebuah voxel akan diuji apakah bertabrakan dengan *face* dari objek. Apabila voxel tersebut tidak bertabrakan, maka *voxel* tersebut tidak lagi digunakan (*unused node*). Sebaliknya, apabila *voxel* tersebut bertabrakan, maka *voxel* tersebut akan dipecah lagi menjadi bagian-bagian yang lebih kecil, kecuali *voxel* tersebut sudah memiliki *size* terkecil sesuai input pengguna. Fungsi ini menerapkan uji-uji dengan menerapkan *Separating Axis Theorem*. Berdasarkan SAT, kedua benda dapat diuji apakah saling bertabrakan atau tidak, dengan menguji normal dari masing-masing benda, lalu menguji apakah ada bagian dari benda lainnya yang menyentuh *axis* dari benda tersebut. Pada kasus proyek ini, dua benda yang diuji adalah *voxel* dan *face* (3D dan 2D). Untuk memudahkan perhitungan, *voxel* dan *face* yang diuji ditransformasi ke pusat ruang (0,0,0). Fungsi menerima parameter *centre* berupa pusat dari *voxel*, *halfSide* berupa setengah panjang dari *voxel*, dan *f* berupa bentuk *face* yang ingin diuji.

  - ➢ Pertama, kita menguji apakah ada *face* yang bertabrakan dengan setiap normal dari *voxel*. Kita perlu menganggap bahwa sebuah *face* merupakan kubus yang terbentuk dari ketiga titik yang membentuknya, kita hanya perlu menguji apakah ada salah satu saja, dari komponen X, Y, atau Z dari *face* yang masuk ke dalam range *voxel*. Apabila ada, maka dilanjutkan ke tes selanjutnya. Jika tidak ada, maka dapat dijamin bahwa kedua objek tidak menyentuh satu sama lain (maka *return false*)
  - ➢ Kedua, kita menguji apakah *voxel* menyentuh bidang 2D yang dibentuk oleh *face*. Cara pengujian dilakukan dengan menghitung jarak dari bidang ke pusat ruang (0,0,0) dikurangi dengan jarak dari titik sudut *voxel* dengan pusat ruang (0,0,0). Jika hasilnya nol atau negatif, maka mereka saling menyentuh dan masih ada kemungkinan kecil bahwa sebenarnya mereka tidak saling menyentuh. Jika hasilnya positif, maka kedua benda tidak saling menyentuh, dan dapat dijamin bahwa mereka tidak *intersect*.

```go
// == Test sisanya (smua cross biar gd gap yg miss)
var min_p, max_p, r float32

// line 0
min_p = min(p0.Y*l0.Z - p0.Z*l0.Y, p2.Y*l0.Z - p2.Z*l0.Y) // x & l0
max_p = max(p0.Y*l0.Z - p0.Z*l0.Y, p2.Y*l0.Z - p2.Z*l0.Y)
r = halfSide * (abs(l0.Z) + abs(l0.Y))
if min_p > r || max_p < -r { return false }

min_p = min(p0.Z*l0.X - p0.X*l0.Z, p2.Z*l0.X - p2.X*l0.Z) // y & l0
max_p = max(p0.Z*l0.X - p0.X*l0.Z, p2.Z*l0.X - p2.X*l0.Z)
r = halfSide * (abs(l0.Z) + abs(l0.X))
if min_p > r || max_p < -r { return false }

min_p = min(p1.X*l0.Y - p1.Y*l0.X, p2.X*l0.Y - p2.Y*l0.X) // z & l0
max_p = max(p1.X*l0.Y - p1.Y*l0.X, p2.X*l0.Y - p2.Y*l0.X)
r = halfSide * (abs(l0.Y) + abs(l0.X))
if min_p > r || max_p < -r { return false }

// line 1
min_p = min(p0.Y*l1.Z - p0.Z*l1.Y, p1.Y*l1.Z - p1.Z*l1.Y) // x & l1
max_p = max(p0.Y*l1.Z - p0.Z*l1.Y, p1.Y*l1.Z - p1.Z*l1.Y)
r = halfSide * (abs(l1.Z) + abs(l1.Y))
if min_p > r || max_p < -r { return false }
```

```go
min_p = min(p0.Z*l1.X - p0.X*l1.Z, p1.Z*l1.X - p1.X*l1.Z) // y & l1
max_p = max(p0.Z*l1.X - p0.X*l1.Z, p1.Z*l1.X - p1.X*l1.Z)
r = halfSide * (abs(l1.Z) + abs(l1.X))
if min_p > r || max_p < -r { return false }

min_p = min(p0.X*l1.Y - p0.Y*l1.X, p1.X*l1.Y - p1.Y*l1.X) // z & l1
max_p = max(p0.X*l1.Y - p0.Y*l1.X, p1.X*l1.Y - p1.Y*l1.X)
r = halfSide * (abs(l1.Y) + abs(l1.X))
if min_p > r || max_p < -r { return false }

// line 2
min_p = min(p0.Y*l2.Z - p0.Z*l2.Y, p1.Y*l2.Z - p1.Z*l2.Y) // x & l2
max_p = max(p0.Y*l2.Z - p0.Z*l2.Y, p1.Y*l2.Z - p1.Z*l2.Y)
r = halfSide * (abs(l2.Z) + abs(l2.Y))
if min_p > r || max_p < -r { return false }

min_p = min(p0.Z*l2.X - p0.X*l2.Z, p1.Z*l2.X - p1.X*l2.Z) // y & l2
max_p = max(p0.Z*l2.X - p0.X*l2.Z, p1.Z*l2.X - p1.X*l2.Z)
r = halfSide * (abs(l2.Z) + abs(l2.X))
if min_p > r || max_p < -r { return false }

min_p = min(p0.X*l2.Y - p0.Y*l2.X, p1.X*l2.Y - p1.Y*l2.X) // z & l2
max_p = max(p0.X*l2.Y - p0.Y*l2.X, p1.X*l2.Y - p1.Y*l2.X)
r = halfSide * (abs(l2.Y) + abs(l2.X))
if min_p > r || max_p < -r { return false }
```

Gambar 2.11 Source Code intersect.go (3)

➢ Ketiga, kita normal dari setiap *edge* (sisi) *voxel* dengan setiap *edge* dari *face*. Tujuan dari tahap terakhir ini adalah memastikan bahwa objek tidak saling menyentuh dari segala sudut normal lainnya (yang bukan X, Y, Z, dan normal bidang *face*)

- outhandle.go
  Source Code:

```go
package voxelizer

import (
    "fmt"
    "math"
    "os"
    "tucil/src/octree"
)

func GetLeaves(node *octree.OctreeNode, leaves *[]*octree.OctreeNode) {
    if node == nil {
        return
    }

    if node.IsLeaf {
        *leaves = append(*leaves, node)
        return
    }

    for i := range 8 {
        GetLeaves(node.Children[i], leaves)
    }
}

func PrintDepthDetails() {
    fmt.Println("--- Octree Depth ---")
    fmt.Printf("n : %v\n\n", len(DepthList)-1)

    fmt.Println("--- Used Nodes ---")
    for i := range len(DepthList) {
        fmt.Printf("Depth - %v : %v\n", i, DepthList[i])
    }
    fmt.Println()

    fmt.Println("--- Unused Nodes ---")
    for i := range len(DepthList) {
        fmt.Printf("Depth - %v : %v\n", i, int(math.Pow(8, float64(i)))-DepthList[i])
    }
    fmt.Println()
}
```

Gambar 2.12 Source Code outhandle.go (1)

- Prosedur GetLeaves
  Prosedur ini menerima input berupa node octree paling pertama, dan juga pointer untuk menyimpan array seluruh *leaves* yang merupakan hasil dari proses divcon.

Prosedur melakukan rekursi untuk memperoleh seluruh *leaf* yang telah terbentuk dalam proses *divide and conquer*. Apabila *node.IsLeaf* benar, maka ia merupakan unit *voxel* terkecil, yang akan ditambahkan dalam array '*leaves*'

- Prosedur PrintDepthDetails

  Prosedur ini melakukan *printing* data-data yang penting dalam proyek ini, seperti kedalaman *octree*, jumlah *node* yang dipakai dan yang tidak dipakai.

```go
func ExportToOBJ(filename string, leaves []*octree.OctreeNode) error {
    file, err := os.Create(filename)
    if err != nil {
        return err
    }
    defer file.Close()

    fmt.Fprintf(file, "# Voxels by Vinli Owen\n\n")

    for _, leaf := range leaves {
        cx, cy, cz := leaf.Centre.X, leaf.Centre.Y, leaf.Centre.Z
        h := leaf.HalfSide

        fmt.Fprintf(file, "v %f %f %f\n", cx-h, cy-h, cz+h) // bot l
        fmt.Fprintf(file, "v %f %f %f\n", cx+h, cy-h, cz+h) // bot r
        fmt.Fprintf(file, "v %f %f %f\n", cx+h, cy-h, cz-h) // top r
        fmt.Fprintf(file, "v %f %f %f\n", cx-h, cy-h, cz-h) // top l

        fmt.Fprintf(file, "v %f %f %f\n", cx-h, cy-h, cz-h) // bot l
        fmt.Fprintf(file, "v %f %f %f\n", cx+h, cy-h, cz-h) // bot r
        fmt.Fprintf(file, "v %f %f %f\n", cx+h, cy-h, cz-h) // top r
        fmt.Fprintf(file, "v %f %f %f\n", cx-h, cy-h, cz-h) // top l
    }
```

```go
    fmt.Fprintf(file, "\n")

    fmt.Println("--- Vertices Generated ---")
    fmt.Printf("v : %v\n\n", len(leaves)*8)

    // ini urutan rotasiny hrs pas, intiny buat facesny
    v := 1
    for i := 0; i < len(leaves); i++ {
        fmt.Fprintf(file, "f %d %d %d %d\n", v, v+1, v+2, v+3)   // front
        fmt.Fprintf(file, "f %d %d %d %d\n", v+5, v+4, v+7, v+6) // back
        fmt.Fprintf(file, "f %d %d %d %d\n", v+3, v+2, v+6, v+7) // top
        fmt.Fprintf(file, "f %d %d %d %d\n", v+4, v+5, v+1, v)   // bot
        fmt.Fprintf(file, "f %d %d %d %d\n", v+1, v+5, v+6, v+2) // r
        fmt.Fprintf(file, "f %d %d %d %d\n", v+4, v, v+3, v+7)   // l

        v += 8
    }

    fmt.Println("--- Faces Generated ---")
    fmt.Printf("f : %v\n\n", len(leaves)*6)

    return nil
}
```

Gambar 2.13 Source Code outhandle.go (2)

- Prosedur ExportToObj

  Prosedur ini menuliskan seluruh hasil *leaves* yang telah terbentuk dalam proses *divide and conquer* ke dalam *file* output 'filename'. Dikarenakan file output berupa .obj, maka agar dapat dirender, hasil dari seluruh *voxel* harus dibuat dalam bentuk kubus. Karena itu, pertama-tama, untuk seluruh *voxel*, dibentuk 8 titik sudut yang akan membangun kerangka kubus. Selanjutnya, untuk seluruh *voxel*, akan dibentuk masing-masing enam *face*. Dengan begitu, maka seluruh *face* akan menutup dan membentuk sebuah kubus.

e. Package wrapper

- Wrapper.go

  Source code:

```go
package wrapper

import (
    "math"
    "tucil/src/octree"
    "tucil/src/point"
)

func WrapInBox(arrPoint []point.Point) (oct *octree.OctreeNode) {
    oct = nil
    var max_x, max_y, max_z float32 = -math.MaxFloat32, -math.MaxFloat32, -math.
    MaxFloat32

    var min_x, min_y, min_z float32 = -max_x, -max_y, -max_z

    // cari titik titik minimum maksimum untuk dibuat balok yang mengelilingi
    for v := range arrPoint {
        x := arrPoint[v].X
        y := arrPoint[v].Y
        z := arrPoint[v].Z
```

```go
        if x > max_x {
            max_x = x
        }
        if x < min_x {
            min_x = x
        }

        if y > max_y {
            max_y = y
        }
        if y < min_y {
            min_y = y
        }

        if z > max_z {
            max_z = z
        }
        if z < min_z {
            min_z = z
        }
    }

    var sd, cx, cy, cz float32 //s: panjang, c: tengah, sd: stengah panjang maks
    // ceil biar ga gmpg error
    cx = float32(math.Ceil(float64((max_x + min_x) / 2)))
    cy = float32(math.Ceil(float64((max_y + min_y) / 2)))
    cz = float32(math.Ceil(float64((max_z + min_z) / 2)))

    var maxDist float32 = 0
    for i := range arrPoint {
        dx := float32(math.Abs(float64(arrPoint[i].X - cx)))
        dy := float32(math.Abs(float64(arrPoint[i].Y - cy)))
        dz := float32(math.Abs(float64(arrPoint[i].Z - cz)))
```

```go
        if dx > maxDist {
            maxDist = dx
        }
        if dy > maxDist {
            maxDist = dy
        }
        if dz > maxDist {
            maxDist = dz
        }
    }
    sd = maxDist

    // biar kelipatan dua aj
    var exp float64 = 0
    for float32(math.Pow(2, exp)) < sd {
        exp++
    }
    sd = float32(math.Pow(2, exp))

    return octree.NewOctreeNode(point.Point{X: cx, Y: cy, Z: cz}, sd)
}
```

Gambar 2.14 Source Code Wrapper.go

- Fungsi WrapInBox
  Suatu fungsi yang menerima parameter array of Point arrPoint dan mengembalikan pointer ke OctreeNode.
  Fungsi ini akan menghitung batas minimal dan maksimal untuk setiap X, Y, dan Z untuk membentuk kubus paling minimal yang membungkus semua titik objek. Agar memudahkan perhitungan nantinya, ukuran kubus dibuat hasil perpangkatan 2.

f. Package main
   - main.go
     Source code:

```go
package main

import (
    "fmt"
    "log"
    "math"
    "time"
    "tucil/src/octree"
    "tucil/src/parser"
    "tucil/src/point"
    "tucil/src/voxelizer"
)

func main() {
    var filename string
    var targetSize float64

    // Input
    fmt.Println("\n=== MAMBA VOXELIZER ===")
    fmt.Println("Please enter the filename you desire")
    fmt.Println("Hint : file located in tc/")
    fmt.Print(">> ")
    fmt.Scanf("%s\n", &filename)

    targetSize = math.Pow(2, targetSize)

    // Parse
    vertices, faces := parser.Parse("tc/" + filename)

    var arrPoints []point.Point
    arrPoints = point.ToPoint(vertices)

    var arrFaces []point.Face
    arrFaces = point.ToFace(arrPoints, faces)

    fmt.Println("\n=== STARTS VOXELIZING ===")
```

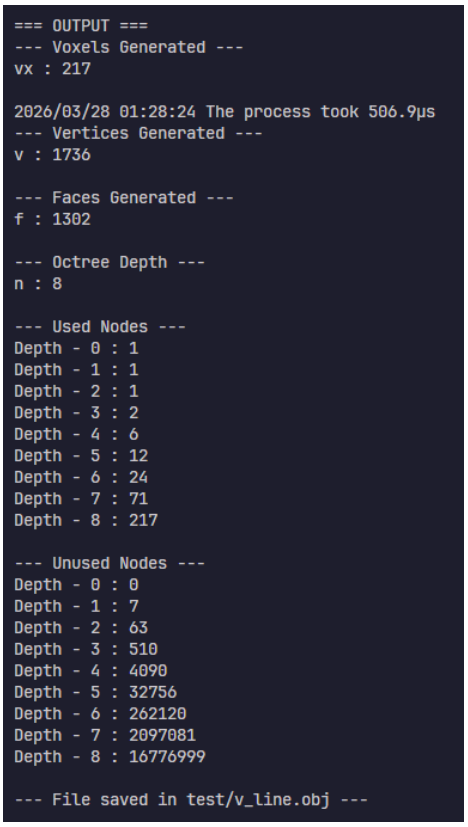
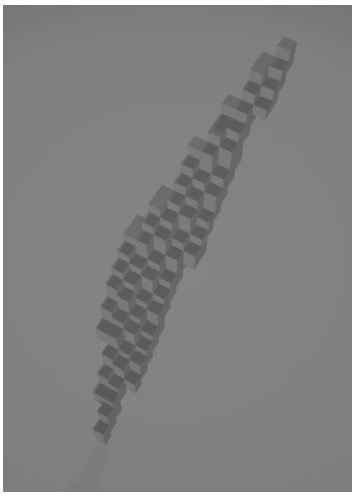
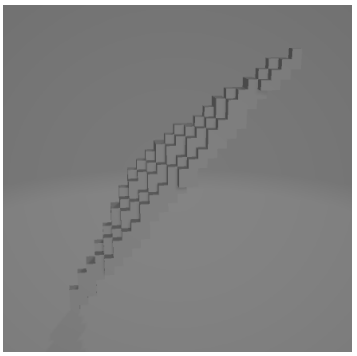
```go
    rootNode, start_time := voxelizer.StartVoxelize(arrPoints, arrFaces)

    fmt.Println("\n=== OUTPUT ===")
    var leaves []*octree.OctreeNode
    voxelizer.GetLeaves(rootNode, &leaves)

    fmt.Println("--- Voxels Generated ---")
    fmt.Printf("vx : %v\n\n", len(leaves))

    elapsed := time.Since(start_time)
    log.Printf("The process took %s", elapsed)

    // print vertex & face
    outputFile := "test/v_" + filename
    errr := voxelizer.ExportToOBJ(outputFile, leaves)
    if errr != nil {
        log.Fatalf("Failed to save voxels: %v", errr)
    }

    // print nodes detail
    voxelizer.PrintDepthDetails()

    fmt.Printf("--- File saved in %s ---\n\n", outputFile)

}
```

Gambar 2.15 Source Code main.go

- Fungsi main
  Fungsi ini menerima input di terminal untuk nama file, lalu memproses objek dengan urutan:
  - Parse isi file .obj
  - Menyimpan face dan vertex dengan tipe data yang sesuai
  - Memanggil fungsi StartVoxelize
  - Menyimpan semua leaves
  - Menunjukkan waktu proses voxelization (tidak termasuk input dan output)
  - Melakukan output informasi jumlah vertex, face, kedalaman octree, node yang terpakai, serta node yang tidak terpakai
  - Membuat output ke file .obj yang sesuai

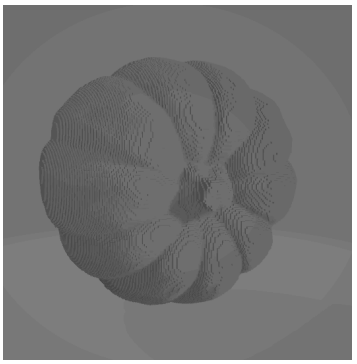
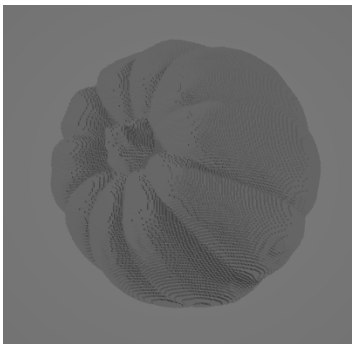
3 Contoh Tangkapan Layar

Pada proses ini digunakan level 1 untuk menunjukkan proses yang paling mencerminkan brute force.

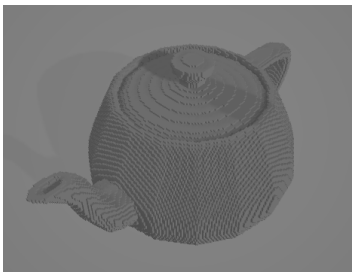
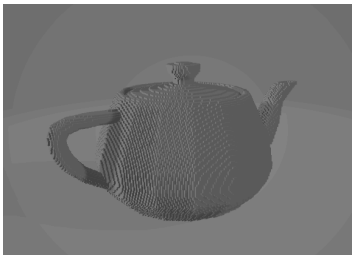
No	Input	Potongan Proses Eksekusi	Hasil akhir
1	line.obj	 === OUTPUT === --- Voxels Generated --- vx : 217 2026/03/28 01:28:24 The process took 506.9µs --- Vertices Generated --- v : 1736 --- Faces Generated --- f : 1302 --- Octree Depth --- n : 8 --- Used Nodes --- Depth - 0 : 1 Depth - 1 : 1 Depth - 2 : 1 Depth - 3 : 2 Depth - 4 : 6 Depth - 5 : 12 Depth - 6 : 24 Depth - 7 : 71 Depth - 8 : 217 --- Unused Nodes --- Depth - 0 : 0 Depth - 1 : 7 Depth - 2 : 63 Depth - 3 : 510 Depth - 4 : 4090 Depth - 5 : 32756 Depth - 6 : 262120 Depth - 7 : 2097081 Depth - 8 : 16776999 --- File saved in test/v_line.obj ---	 

<table>
<tr>
<td>2</td>
<td>pumpkin.obj</td>
<td>

```
=== OUTPUT ===
--- Voxels Generated ---
vx : 114293

2026/03/28 01:29:38 The process took 65.4976ms
--- Vertices Generated ---
v : 914344

--- Faces Generated ---
f : 685758

--- Octree Depth ---
n : 8

--- Used Nodes ---
Depth - 0 : 1
Depth - 1 : 8
Depth - 2 : 32
Depth - 3 : 122
Depth - 4 : 419
Depth - 5 : 1770
Depth - 6 : 7112
Depth - 7 : 28607
Depth - 8 : 114293

--- Unused Nodes ---
Depth - 0 : 0
Depth - 1 : 0
Depth - 2 : 32
Depth - 3 : 390
Depth - 4 : 3677
Depth - 5 : 30998
Depth - 6 : 255032
Depth - 7 : 2068545
Depth - 8 : 16662923

--- File saved in test/v_pumpkin.obj ---
```

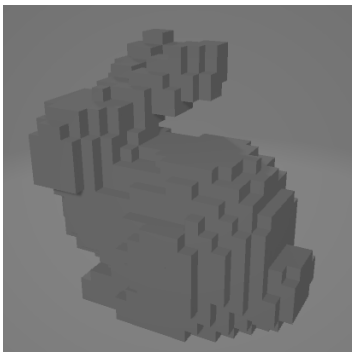
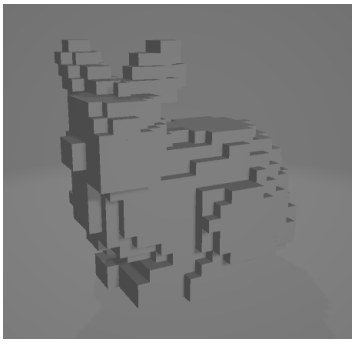
</td>
<td>


</td>
</tr>
<tr>
<td>3</td>
<td>teapot.obj</td>
<td>

```
=== OUTPUT ===
--- Voxels Generated ---
vx : 60150

2026/03/28 01:30:15 The process took 36.4522ms
--- Vertices Generated ---
v : 481200

--- Faces Generated ---
f : 360900

--- Octree Depth ---
n : 8

--- Used Nodes ---
Depth - 0 : 1
Depth - 1 : 8
Depth - 2 : 16
Depth - 3 : 56
Depth - 4 : 210
Depth - 5 : 892
Depth - 6 : 3714
Depth - 7 : 15091
Depth - 8 : 60150

--- Unused Nodes ---
Depth - 0 : 0
Depth - 1 : 0
Depth - 2 : 48
Depth - 3 : 456
Depth - 4 : 3886
Depth - 5 : 31876
Depth - 6 : 258430
Depth - 7 : 2082061
Depth - 8 : 16717066

--- File saved in test/v_teapot.obj ---
```

</td>
<td>


</td>
</tr>
</table>

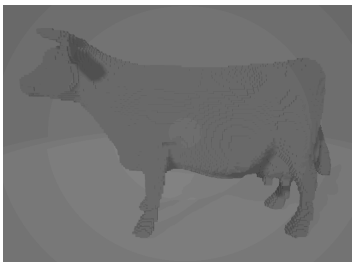
<table>
<tr>
<td>4</td>
<td>bunny.obj</td>
<td>

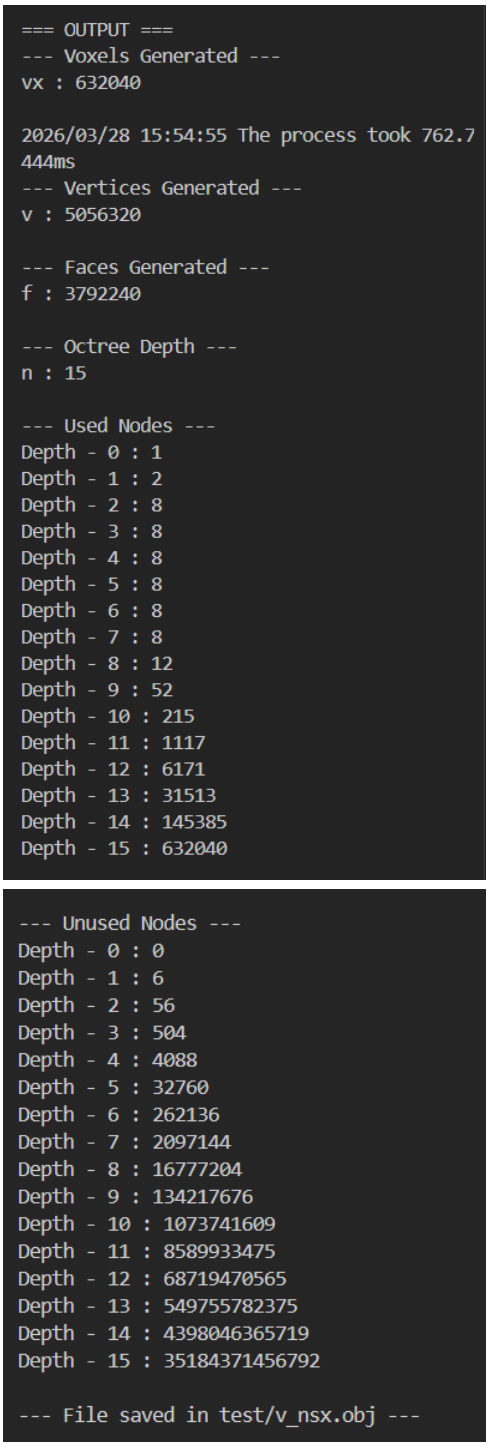
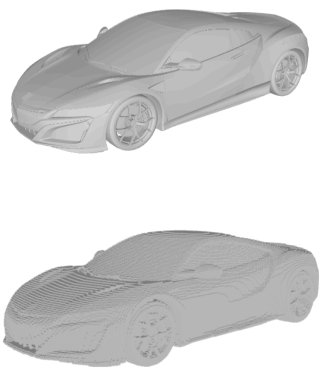
```
=== OUTPUT ===
--- Voxels Generated ---
vx : 1306

2026/03/28 01:30:34 The process took 15.5716ms
--- Vertices Generated ---
v : 10448

--- Faces Generated ---
f : 7836

--- Octree Depth ---
n : 8

--- Used Nodes ---
Depth - 0 : 1
Depth - 1 : 4
Depth - 2 : 4
Depth - 3 : 4
Depth - 4 : 8
Depth - 5 : 18
Depth - 6 : 74
Depth - 7 : 304
Depth - 8 : 1306

--- Unused Nodes ---
Depth - 0 : 0
Depth - 1 : 4
Depth - 2 : 60
Depth - 3 : 508
Depth - 4 : 4088
Depth - 5 : 32750
Depth - 6 : 262070
Depth - 7 : 2096848
Depth - 8 : 16775910

--- File saved in test/v_bunny.obj ---
```

</td>
<td>


</td>
</tr>
<tr>
<td>5</td>
<td>cow.obj</td>
<td>

```
=== OUTPUT ===
--- Voxels Generated ---
vx : 39093

2026/03/28 01:30:51 The process took 31.4359ms
--- Vertices Generated ---
v : 312744

--- Faces Generated ---
f : 234558

--- Octree Depth ---
n : 8

--- Used Nodes ---
Depth - 0 : 1
Depth - 1 : 8
Depth - 2 : 14
Depth - 3 : 34
Depth - 4 : 163
Depth - 5 : 615
Depth - 6 : 2389
Depth - 7 : 9673
Depth - 8 : 39093

--- Unused Nodes ---
Depth - 0 : 0
Depth - 1 : 0
Depth - 2 : 50
Depth - 3 : 478
Depth - 4 : 3933
Depth - 5 : 32153
Depth - 6 : 259755
Depth - 7 : 2087479
Depth - 8 : 16738123

--- File saved in test/v_cow.obj ---
```

</td>
<td>


</td>
</tr>
</table>

6	nsx.obj		

4 Checklist

No	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan	v	
2	Program berhasil dijalankan	v	
3	Hasil konversi .obj terdiri dari	v	

	kubus-kubus kecil		
4	Program informasi-informasi atau report	v	
5	Program dapat menerima masukan .obj, dan mengembalikan output	v	
6	[Bonus] Program menggunakan *concurrency*	v	
7	[Bonus] Program dapat menampilkan visualisasi .obj		v

5 Pernyataan Tidak Melakukan Kecurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Vincent Rionarlie Ray Owen Martin

6 Lampiran

Repository dapat diakses pada: https://github.com/Vixrlie/Tucil2_13524031_13524033